

SpendSight: Privacy-Preserving Personal Finance Q&A with Sub-2B LLMs

Anonymous ACL submission

Abstract

People often speculate about their personal finances: “Has my spending on groceries increased compared to past year?”, “How does my spending on clothes look for the past 6 months?”. However, answering them with cloud-hosted LLMs transmits sensitive financial data to external servers, exposing users to data leakage and inferential privacy risks. We present **SpendSight**, a study of fully on-device personal finance Q&A tool calling that keeps all data strictly local. We evaluate seven quantized models (six sub-2B and one 3B) across 115 test cases and five analytical tools, comparing single-agent against dual-agent (router-specialist) architectures. For sub-2B models, dual-agent decomposition yields substantial gains: Qwen3.5-0.8B achieves 89.1% Task Accuracy in 3.0 s with 1.2 GB RAM. A 3B reference model (G9v3-3B, Q4_K_M) achieves 90.6% single-agent Task Accuracy at 2.3 s, suggesting that larger models with aggressive quantization can bypass decomposition. A real-world case study confirms practical impact: a user reduced snack spending by 84.7% (¥38,453 → ¥5,893) after insights surfaced entirely on-device.

1 Introduction

People often wonder about their personal finances: “Has my spending on dining increased month over month? What were some of my largest purchases last month? These are natural, everyday questions and modern large language models (LLMs) can answer them by calling structured analytical tools over a user’s expense database (Schick et al., 2023; Patil et al., 2023; Qin et al., 2024).

The problem is *where* this inference happens. Cloud-hosted assistants require users to transmit their financial queries, and often the underlying data, to external servers. A query like “how much did I spend on therapy last month?” is not merely a database lookup; it is a disclosure of a health

condition, a temporal pattern, and a financial commitment. Financial records encode sensitive behavioral patterns that risk data leakage and inferential privacy violations when processed remotely (Staab et al., 2024; Yao et al., 2024).

This creates an underexplored problem at the intersection of financial NLP and on-device inference: *can sub-2B quantized models running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial natural language?* If so, on-device inference becomes not merely a technical curiosity but a practical privacy guarantee for real users, one that eliminates cloud dependency entirely.

Existing work on financial tool calling, such as FinMCP-Bench (Zhu et al., 2026) and FinTrace (Cao et al., 2026), evaluates cloud-scale models ($\geq 7B$) on multi-step trajectories. General-purpose on-device models like Octopus v2 (Chen and Li, 2024) and Hammer (Lin et al., 2024) demonstrate the potential of smaller models but have not been evaluated on domain-specific financial schemas or under consumer-hardware latency constraints. Our study bridges this gap.

We address three research questions (RQ):

- **RQ1:** Can on-device inference with sub-2B models provide practically viable, private personal finance analytics (i.e., can users obtain accurate personal finance insights without sending query data to cloud providers)?
- **RQ2:** Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy gains over a joint single-agent architecture?
- **RQ3:** What are the primary failure modes across architectures, and how do they differ in the real-world harms they produce for users relying on financial outputs?

This paper presents the following contributions:

1. A **study of on-device financial tool calling**: Qwen3.5-0.8B dual achieves 89.1% Task Accuracy at 3.0 s latency and ≈ 1.2 GB RAM, keeping all data strictly local.
2. A **fine-grained error taxonomy** revealing that dual-agent architectures shift errors from silent parameter corruption toward visible omissions, a qualitative safety improvement for financial outputs.
3. A **comparative evaluation** of single-agent vs. dual-agent across six sub-2B quantized models (0.8B–2.0B), showing dual-agent reduces latency (up to 48%), improves exact-match correctness (+17.4 pp CR), and stabilises sub-1B models.
4. An **extended comparison** with a 3B model (G9v3-3B, Q4_K_M) showing that larger models under aggressive quantization can match sub-2B dual-agent accuracy in single-agent mode, delineating when decomposition is necessary.
5. A **real-world case study**: on-device insights led a user to reduce snack spending by 84.7% over six months, with no data leaving the device.

2 On-Device Inference as Privacy Protection

Conversational interfaces for personal finance expose a structural issue: routing natural-language queries through cloud LLMs leaks behavioral signals that can be retained, aggregated, and exploited (Staab et al., 2024; Yao et al., 2024). SpendSight addresses this threat by executing all inference *on-device*. By leveraging sub-2B quantized models that run efficiently on consumer hardware (tested on an Apple M1 with 8 GB memory), all user data, queries, and model outputs remain strictly local, preserving *contextual integrity* (Nissenbaum, 2004).

The threat model encompasses cloud data retention, query-level inference attacks, and third-party data exposure. On-device inference eliminates all three by ensuring queries never leave local infrastructure. While sub-2B models have historically been unreliable for structured tool calling (Kavathekar et al., 2025), our study demonstrates that architectural decomposition (separating intent classification from parameter extraction) bridges

this reliability gap, making private on-device inference practical for real personal financial Q&A systems.

3 Related Work

Tool-Augmented Language Models. Toolformer (Schick et al., 2023) demonstrated self-supervised tool-use learning; Gorilla (Patil et al., 2023) and ToolLLM (Qin et al., 2024) scaled to thousands of real-world APIs. HuggingGPT (Shen et al., 2023) and ToolAlpaca (Tang et al., 2023) explored multi-step orchestration. APIGen (Liu et al., 2024b) introduced a verified pipeline for generating function-calling training data, showing that 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. These approaches rely on cloud-hosted models (≥ 7 B), leaving the privacy-constrained on-device setting largely unaddressed.

Function-Calling Evaluation. The Berkeley Function Calling Leaderboard (Patil et al., 2025) established a benchmark using AST substring matching and execution-based validation. Kavathekar et al. (2025) proposed an evaluation suite (FSP, ACS, AVC, Task Accuracy, and Correct Ratio) for small model function calling; we adopt these metrics directly. CallNavi (Song et al., 2025) proposed a two-stage router+specialist approach, similar to our dual-agent design, but evaluates models ≥ 7 B on general-purpose APIs. Jhandi et al. (2025) studied fine-tuning SLMs for agentic tool calling using trajectory-level data, demonstrating that targeted domain adaptation can close the gap with larger models. In contrast, we focus on sub-2B models for domain-specific, single-turn personal finance tasks without fine-tuning.

On-Device Function Calling. MobileLLM (Liu et al., 2024a), Phi-3 (Abdin et al., 2024), and related surveys (Xu et al., 2024) have explored sub-4B parameter models for mobile deployment. Octopus v2 (Chen and Li, 2024) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer (Lin et al., 2024) introduced function masking to reduce overfitting to function names, addressing an issue similar to the category normalisation errors we observe. Apple Intelligence (Gunter et al., 2024) validated on-device tool calling at scale with a ~ 3 B model. Ka-

179 [vathekar et al. \(2025\)](#) found that structured output
180 compliance is the primary bottleneck for SLMs.

181 **Financial NLP and Tool Calling.** FinQA ([Chen](#)
182 [et al., 2021](#)) and FinGPT ([Yang et al., 2023](#)) tar-
183 get financial numerical reasoning and generation
184 but focus on text generation rather than structured
185 tool invocation. More recently, FinMCP-Bench
186 ([Zhu et al., 2026](#)) evaluates LLM agents for real-
187 world financial tool use under the Model Context
188 Protocol, and FinTrace ([Cao et al., 2026](#)) provides
189 trajectory-level evaluation of tool calling for long-
190 horizon financial tasks. Both target cloud-scale
191 models ($\geq 7B$) on multi-step trajectories involving
192 dozens of financial APIs. Our study is comple-
193 mentary: we focus on *single-turn, on-device* tool
194 calling with sub-2B models for personal finance,
195 a setting where privacy constraints make cloud in-
196 ference undesirable and where the user directly
197 consumes the output for personal decision-making.
198 Personal finance assistants ([Li et al., 2024](#)) require
199 strict privacy, making local execution crucial.

200 **Privacy for Financial Language Models.** Con-
201 textual integrity ([Nissenbaum, 2004](#)) dictates that
202 personal financial data should not cross contextual
203 norms. Since LLMs can reconstruct sensitive at-
204 tributes from text ([Staab et al., 2024](#)), on-device
205 processing is critical to eliminate cloud exposure
206 pathways entirely.

207 4 System Design

208 4.1 Tool Inventory

209 SpendSight exposes five analytical tools over a
210 user’s expense database. Table 1 summarises each
211 tool and its parameter space. These tools span core
212 personal finance use cases that reflect the questions
213 users naturally ask: time series analysis (“*how has*
214 *my spending changed?*”), categorical breakdown
215 (“*how does the breakdown of my spending look?*”),
216 period comparison (“*how does this month compare*
217 *to last?*”), aggregation (“*how much did I spend on*
218 *X?*”), and ranked retrieval (“*what were my biggest*
219 *purchases?*”).

220 4.2 Architectural Configurations

221 We evaluate two local architectures for mapping
222 natural language user queries to tool calls.

223 **Single-Agent Pipeline.** The single-agent
224 pipeline uses a single prompt with all five tool
225 definitions, parameter schemas, and joint few-shot
226 examples. The model must select the tool and

Tool	ID	Key Params
plot_time_series	1	category, year/month, ranges, months, ignore_rent
plot_distribution	2	category, year/month/day, ranges, months, ignore_rent
plot_comparison_bars	3	category, y1/m1, y2/m2, sm/em/ey, ignore_rent
calculate_total	4	category, year/month/day, remarks, months, ignore_rent
get_top_expenses	5	n, category, year/month/day, months, min_amount, ignore_rent

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

227 extract parameters in a single pass, outputting a
228 JSON object containing the tool ID (1–5) and argu-
229 ments for the chosen tool. The prompt includes
230 structured output rules, date-handling instructions,
231 and 73 tool-specific examples aggregated across
232 all 5 tools. Prompt length: $\approx 5,500$ tokens (context
233 window inclusive).

234 **Dual-Agent Pipeline.** The task is decomposed
235 into two sequential inference calls:

- 236 1. **Router:** A focused intent-classification
237 prompt with 19 few-shot examples and ex-
238 plicit disambiguation rules (e.g., “compare”
239 requires two distinct time periods) elicits a
240 single digit (1–5).
- 241 2. **Specialist:** The predicted tool ID selects a
242 tool-specific prompt containing 13–16 exam-
243 ples for that *one* tool, along with detailed pa-
244 rameter definitions and date-handling rules.

245 The router sees $\approx 1,000$ tokens; the specialist sees
246 $\approx 1,500$ – $2,000$ tokens with deeper, more directly
247 relevant supervision. This simplifies the task for
248 each model call and provides more relevant in-
249 context examples, though it requires two sequen-
250 tial inference passes. Combined prompt tokens:
251 $\approx 2,700$ per query. Both architectures inject a dy-
252 namically generated category vocabulary—11 ma-
253 jor categories and ≈ 60 subcategory aliases drawn
254 from the user’s own dataset (Appendix B)—into
255 a {metadata} block within the prompt, ground-
256 ing category parameter extraction in the user’s ac-
257 tual expense taxonomy. These category names
258 are inherently sensitive: they reveal what a user
259 spends money on (e.g., healthcare, dining, dona-
260 tions), making their transmission to third-party
261 cloud providers a privacy risk. In dual-agent mode,
262 only the specialist receives this vocabulary; the
263 router operates on pure intent classification without

Task Group	Cases	Avg Params	Avg C
Time Series	23	2.9	7.0
Distribution	21	2.5	6.4
Comparison	28	4.4	8.8
Calculate Total	22	2.4	6.3
Top Expenses	21	3.2	6.7
Total	115	3.2	7.1

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

distance; $d_{\text{rel}} \in \{0, 1\}$ flags relative time expressions; $d_{\text{multi}} \in \{0-2\}$ counts compound parameter groups; $d_{\text{abbr}} \in \{0, 1\}$ flags abbreviated dates; and $d_{\text{hol}} \in \{0, 2\}$ flags calendar knowledge requirements. Scores are binned into three tiers: **L1** ($C \leq 4$, 24 cases), **L2** ($5 \leq C \leq 7$, 45 cases), **L3** ($C \geq 8$, 46 cases).

5.3 Evaluation Metrics

We adopt the metric suite of Kavathekar et al. (2025) to assess tool routing and parameter extraction performance:

- **FSP** (Function Selection Precision): Binary indicator of correct tool prediction (i.e. routing).
- **ACS** (Argument Completeness Score): F_1 on argument *name* overlap (ignoring values).
- **AVC** (Argument Value Correctness): Fraction of correctly named arguments whose extracted values also match.
- **Task Accuracy**: F_1 on the combined set $\{\text{tool}\} \cup \{k=v\}$, jointly capturing tool selection and parameter correctness.
- **Correct Ratio (CR)**: Strict binary indicator: 1.0 only when Task Acc = 1.0, i.e. the entire function call is perfectly correct for execution.

4.3 Post-Hoc Parameter Validation

A post-processing validation layer corrects category names deterministically without additional model calls:

1. **Exact match** against the known vocabulary.
2. **Morphological normalisation**:
groceries $\rightarrow$ grocery, cafés $\rightarrow$ cafe.
3. **Edit-distance matching** via difflib (cutoff ≥ 0.80).
4. **Token overlap scoring** (Jaccard; cutoff ≥ 0.50).

All results report both raw (pre-validation) and validated (post-validation) metrics.

5 Evaluation Design

5.1 Test Suite

The evaluation comprises 115 test cases across five task groups (Table 2). Queries are written in natural and informal language: abbreviations (“compare electricity ’24 vs ’25”), colloquialisms (“can you tell me”), misspellings (“distribution”), and domain-specific terms (“combinis”, “nomikais” “shinkansen”). This mimics how real users interact with personal finance assistants, using casual language with abbreviations and cultural terminology specific to their context.

5.2 Complexity Scoring

Each test case is assigned a difficulty score:

$$C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

where n_p = number of expected parameters; $d_{\text{date}} \in \{0-4\}$ scores date specification complexity; $d_{\text{cat}} \in \{0-2\}$ is the category normalisation

All metrics are averaged across 5 repetitions per test case (temperature = 0.1). 95% bootstrap confidence intervals (Efron, 1979) ($n = 2000$ resamples) were computed for statistical testing of the overall gap between models, rather than reported per-cell.

5.4 Error Taxonomy

Each prediction is classified into one of seven mutually exclusive error types: CORRECT, TOOL_WRONG, PARSE_FAILURE, PARAM_MISSING, PARAM_VALUE_WRONG, PARAM_EXTRA, FORMAT_VIOLATION. This taxonomy is crucial for evaluating the *safety profile* of on-device financial assistants: in personal finance, a silently wrong answer (from a spurious parameter) is more harmful than a visibly incomplete one (from a missing parameter).

Model	Family	Params	RAM
EXAONE-4.0-1.2B	EXAONE-4.0 (2025)	1.2B	2.0 GB
Gemma-3-1B-it	Gemma-3 (2025)	1B	1.6 GB
LFM2.5-1.2B	LFM2.5 (2026)	1.2B	1.4 GB
MiniCPM5-1B	MiniCPM5 (2026)	1B	1.5 GB
Qwen3.5-0.8B	Qwen3.5 (2026)	0.8B	1.3 GB
Qwen3.5-2B	Qwen3.5 (2026)	2B	2.1 GB
G9v3-3B [†]	G9v3 (2026)	3B	2.4 GB

Table 3: Models evaluated. Sub-2B models use Q8_0 GGUF quantization; [†]G9v3-3B uses Q4_K_M. All run via llama.cpp. RAM is average peak RSS.

6 Experimental Setup

6.1 Models

All sub-2B models (Table 3) use Q8_0 GGUF quantization; G9v3-3B uses Q4_K_M. All seven models run via llama.cpp (Gerganov, 2023) with full GPU offload on Apple Metal. G9v3-3B was selected as the top-ranked open-weight model under 4B parameters on the Artificial Analysis Intelligence Index at the time of evaluation. Since G9v3-3B supports an optional extended thinking mode, we disabled it (`enable_thinking=false`) for all runs, to keep it comparable to the non-reasoning inference setting used for the other six models.

6.2 Hardware and Inference Protocol

Experiments run on an Apple M1 (8-core CPU, 8-core GPU, 8 GB unified memory). Parameters: temperature = 0.1, min_p = 0.05, n_ctx = 8192 stop tokens ["\n\n", " "]. Each model is warmed up through all six prompt templates (one router + five specialists) before evaluation begins. Each of the 115 test cases is run 5 times per model per architecture, yielding $115 \times 5 \times 7 \times 2 = 8,050$ total observations.

7 Results

7.1 Overall Performance

Table 4 presents the full results. Figure 1 shows the accuracy-latency trade-off and the task footprint across architectures.

Key findings (RQ1, RQ2). (1) Sub-2B models can reliably call financial tools on consumer hardware (RQ1), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This demonstrates that users can obtain accurate financial an-

alytics, from spending trends to category breakdowns, without any data leaving their device. (2) While single-agent Qwen3.5-2B reaches 92.0% Task Accuracy, dual-agent decomposition is preferred for two reasons (RQ2): (a) Strict correctness: Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, representing a significant reduction in silent parameter errors that could produce incorrect financial outputs a user might act on. (b) Enabling smaller models: Qwen3.5-0.8B improves from 79.8% to 89.1% Task Accuracy (+9.3 pp) while almost halving latency (5.8 s to 3.0 s), making sub-1B models viable for low-memory devices. (3) The 3B reference model G9v3-3B is the sole model where single-agent outperforms dual-agent (90.6% vs. 88.2% Task Accuracy; 71.7% vs. 68.3% CR), indicating a capacity threshold beyond which decomposition provides diminishing returns.

7.2 Per-Tool-Group Analysis

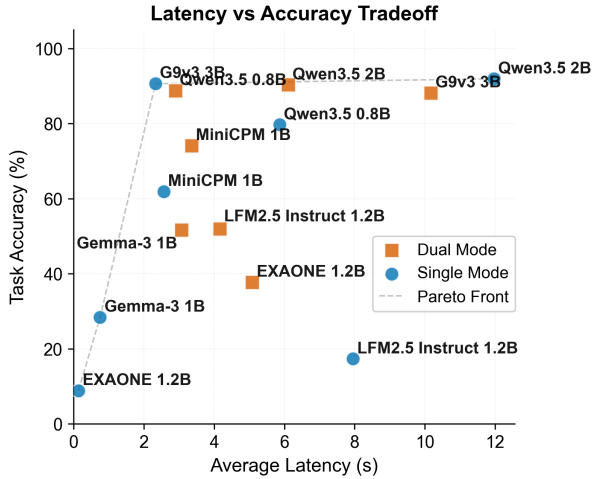
Table 5 shows per-group accuracy for the top two dual-mode models (see Figure 5 in the Appendix for the full model $\times$ category heatmap). `get_top_expenses` reaches 98.9% accuracy with Qwen3.5-2B due to its simple parameter structure, aligning with its role as one of the most common user questions (“what were my biggest purchases?”). Conversely, `plot_time_series` is the most difficult task (74.4% for Qwen3.5-2B) due to complex date specifications and relative time expressions (e.g., “past X months”) requiring temporal reasoning. G9v3-3B dual follows a similar pattern, achieving 96.1% on distribution but only 67.4% on time series.

7.3 Complexity Tier Analysis

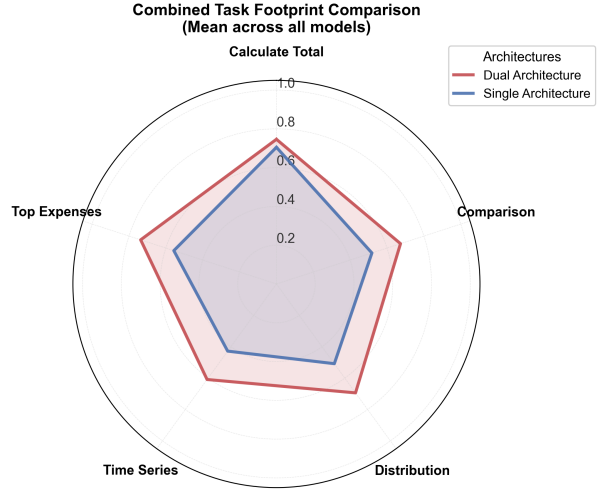
Figure 2 presents the per-model accuracy breakdown across complexity tiers in dual mode. Task Accuracy decreases from 96% (L1) to 86% (L3) for Qwen3.5-2B, and from 91% to 80% for Qwen3.5-0.8B. G9v3-3B dual degrades similarly (96% L1, 82% L3). L3 queries involve constructs like cross-year range comparisons (“compare groceries nov 2024 to april 2025 vs nov 2025 to april 2026”), which require extracting up to 10 parameters with complex temporal constraints, exactly the kind of year-over-year analysis users commonly want.

7.4 Error Taxonomy Analysis

Figure 4 (Appendix) visualises the full error distribution. The error distribution shifts significantly



(a) Accuracy-latency trade-off. Dual-mode Qwen3.5-0.8B (≈ 3 s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at $4\times$ the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 1: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all financial task categories.

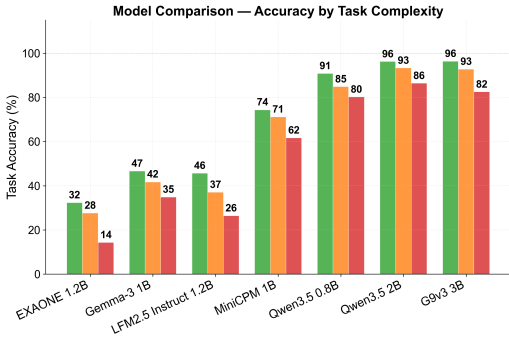


Figure 2: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

439 tectures (11.1% single, 11.8% dual), indicating that
440 the dual-agent error-reduction benefit does not ex-
441 tend to larger models where single-agent capacity
442 is already sufficient.

443 7.5 Token Efficiency and Validation

444 Despite using $\approx 2,700$ prompt tokens per query
445 (vs. $\approx 5,500$ for single-agent), the dual-agent ar-
446 chitecture achieves equivalent or superior accuracy
447 (see Figure 8 in the Appendix). The single-agent
448 pipeline must simultaneously attend to all five tool
449 schemas, while the dual-agent decomposes the
450 context, focusing each pass on a single subtask,
451 achieving higher accuracy per token, an impor-
452 tant property for memory-constrained on-device
453 deployment. The post-hoc validation layer (Sec-
454 tion 4.3) contributes modest but consistent gains
455 (up to +1.2 pp Task Accuracy for MiniCPM5-1B),
456 primarily correcting category casing and morpho-
457 logical normalisation. This confirms that argument
458 extraction, not category matching, remains the pri-
459 mary bottleneck.

460 8 Discussion

Privacy through Task Decomposition. The
dual-agent architecture improves accuracy and
makes local inference viable. By separating routing
(single digit) from extraction (focused JSON), dual-
agent reduces generation complexity per call. De-
composing the task helps sub-2B models perform
reliably on domain-specific financial schemas, mak-

Mode	Model	FSP	ACS	AVC	Task Acc	CR	Latency (s)	RAM (GB)
Single	EXAONE-4.0-1.2B	19.3	N/A	0.3	8.9	0.3	0.1	2.0
	Gemma-3-1B-it	27.0	N/A	47.5	30.0	0.9	0.8	1.6
	LFM2.5-1.2B	28.7	N/A	11.7	17.2	3.1	7.8	1.4
	MiniCPM5-1B	71.1	N/A	86.7	62.3	13.0	2.6	1.5
	Qwen3.5-0.8B	95.0	N/A	91.2	79.8	40.2	5.8	1.3
	Qwen3.5-2B	96.5	N/A	97.6	92.0	60.2	12.0	2.1
Dual	G9v3-3B [†]	95.7	N/A	97.0	90.6	71.7	2.3	2.4
	EXAONE-4.0-1.2B	90.6	3.7	3.3	37.8	3.0	5.0	1.9
	Gemma-3-1B-it	55.1	59.9	71.6	51.7	16.5	3.1	1.5
	LFM2.5-1.2B	69.6	51.6	57.2	51.8	16.3	4.1	1.4
	MiniCPM5-1B	84.9	77.0	81.6	74.3	43.5	3.9	1.5
	Qwen3.5-0.8B	94.3	90.2	96.8	89.1	61.7	3.0	1.2
	Qwen3.5-2B	92.0	91.6	97.4	90.4	77.6	6.2	2.1
	G9v3-3B [†]	92.2	89.7	95.8	88.2	68.3	10.2	2.3

Table 4: Main results (validated metrics, %). **Bold** = best per metric within each mode. FSP = Function Selection Precision; ACS = Argument Completeness Score; AVC = Argument Value Correctness; Task Acc = F₁-based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (N/A) for single-agent configurations. [†]3B model (Q4_K_M), included as a reference point outside the core sub-2B study. Latency and RAM are per-query averages.

Group	Qwen3.5-0.8B		Qwen3.5-2B	
	T.Acc	CR	T.Acc	CR
Time Series	81.6	54.8	74.4	60.9
Distribution	89.6	56.2	93.8	79.0
Comparison	92.5	71.4	93.7	73.6
Calc. Total	87.9	59.1	91.4	84.5
Top Expenses	93.3	64.8	98.9	92.4

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

ing on-device inference a practical privacy guarantee. The exception is G9v3-3B (3B), where single-agent already matches dual-agent performance, indicating a capacity threshold where decomposition becomes unnecessary.

Comparison with Cloud-Scale Tool Calling.

Our study is complementary to cloud-focused financial evaluations like FinMCP-Bench (Zhu et al., 2026) and FinTrace (Cao et al., 2026), which evaluate ≥ 7 B models on multi-step trajectories. SpendSight targets single-turn, on-device, privacy-preserving tool calling where latency must be minimal and no data leaves the device. Our finding that sub-2B models achieve $>89\%$ Task Accuracy suggests cloud-scale models may not be necessary for single-turn personal financial queries.

Model Scale vs. Quantization.

G9v3-3B (3B, Q4_K_M, 2.4 GB) demonstrates an alternative path: a larger model under aggressive quantization. Its 90.6% single-agent Task Accuracy matches sub-2B dual-agent accuracy at 2.3 s latency. However, dual-agent G9v3-3B incurs 10.2 s latency without error-profile improvements. These results suggest a practical threshold around 2–3B parameters be-

yond which single-agent capacity suffices.

Addressing Highly Complex Queries. Even the best model achieves 59.6% strict Correct Ratio on L3 queries (Figure 2), which involve complex cross-year comparisons requiring up to 10 parameters. Fine-tuning on domain-specific synthetic data (Liu et al., 2024b) could improve performance on L3 queries while preserving on-device deployment.

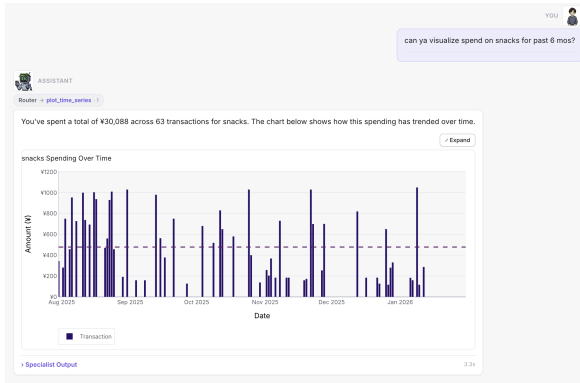
Real-World Impact: From Analytics to Behaviour Change.

Figure 3 illustrates how on-device analytics drive financial behaviour change. In January 2026, a user queried “*can ya visualize spend on snacks for past 6 mos?*” (Figure 3a); Qwen3.5-0.8B dual routed this to `plot_time_series` in 3.3 s, revealing 63 transactions (¥30,088) with peaks near ¥1,000, serving as an alert to cut back. Six months later, the user verified the impact via “*compare spend on snacks jul – dec 2025 vs jan – jun 2026*” (Figure 3b); Qwen3.5-2B dual produced the result in 8.7 s, confirming an **84.7% reduction** (¥38,453 $\rightarrow$ ¥5,893).¹ The per-transaction average increased (¥506 $\rightarrow$ ¥655), confirming fewer purchases resulted in decrease in overall spending. Crucially, all interactions remained strictly on-device.

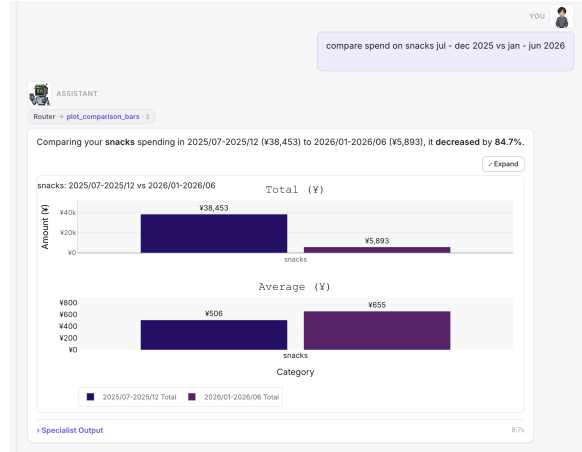
9 Conclusion

We presented SpendSight, a study of privacy-preserving on-device personal finance Q&A tool calling with quantized LLMs. Local inference at this scale is practically reliable: Qwen3.5-0.8B

¹Totals vary slightly between single-period charts and dual-period comparisons due to rolling vs. calendar windows.



(a) **Alert** (Jan 2026): Querying snack spending over past 6 months reveals 63 transactions (¥30,088) with peaks near ¥1,000, prompting a decision to cut back.



(b) **Confirmation** (June 2026): Period comparison confirms an 84.7% reduction in snack spending (¥38,453 → ¥5,893). Higher average transaction (¥506 → ¥655) confirms fewer purchases were the main reason for reduction in spending.

Figure 3: Real-world deployment: private on-device analytics driving behaviour change. Dual-agent Qwen3.5-0.8B (3.3 s) and Qwen3.5-2B (8.7 s) correctly route queries locally. The participant reduced snack spending by 84.7%. Intermediate views in Figures 9–10 (Appendix).

522 dual achieves 89.1% Task Accuracy at 3.0 s latency 552 ments professional financial advice. We advocate
523 on-device. Dual-agent decomposition improves 553 for clear user disclaimers in production.
524 sub-2B strict correctness (60.2% → 77.6% CR) 554 **Data and Code Availability**
525 and shifts errors toward safer PARAM_MISSING 555 Code, prompts, evaluation data, and results will
526 failures, while G9v3-3B reaches 90.6% Task Ac 556 be released upon acceptance via a github reposi-
527 curacy without decomposition. A real-world case 557 tory. Artefacts include: 115 test cases with ground-
528 study confirms tangible impact: a participant re 558 truth labels, router/specialist templates, inference
529 duced snack spending by 84.7% from on-device 559 configs, result CSVs, evaluation runner, plotting
530 insights. We recommend Qwen3.5-0.8B dual for 560 scripts, and model GGUFs for complete repro-
531 memory-constrained settings (≤ 2 GB), Qwen3.5 561 ducibility.
532 2B dual for accuracy-first use (77.6% CR), and
533 G9v3-3B single for minimal-latency single-pass
534 deployment (71.7% CR at 2.3 s).

535 Limitations

536 (1) Sub-2B models use Q8_0 quantisation; G9v3-
537 3B uses Q4_K_M. The interaction between model
538 scale and quantisation warrants further study.
539 (2) Test cases were authored by a single devel-
540 oper in English reflecting a Japan-based context.
541 (3) Ground-truth labels are deterministic. (4) Pri-
542 vacy guarantee is architectural rather than crypto-
543 graphic.

544 Ethics Statement

545 SpendSight preserves user financial privacy via
546 on-device inference. The category taxonomy (Ap-
547 pendix B) and case study (Section 8) derive from
548 author self-tracked expense data, used with consent.
549 Evaluation queries are synthetic; no real transac-
550 tion amounts or records are in the test suite. No
551 third-party data was used. The system comple-

References

- Marah Abidin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, Alon Benham, Misha Bilenko, Johan Borda, Ashley Loren Bucher, and Sébastien Bubeck. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.
- AI9Stars. 2026. G9v3-3b. <https://huggingface.co/ai9stars/G9v3-3B>. Accessed: 2026-08-04.
- Yupeng Cao, Haohang Li, Weijin Liu, Wenbo Cao, Anke Xu, Lingfei Qian, Xiaozhe Peng, Yuxuan Tang, Zeyi Yao, Heng Huang, K. P. Subbalakshmi, Xiao Zhu, Jordan W. Suchow, and Yue Yu. 2026. FinTrace: Holistic trajectory-level evaluation of LLM tool calling for long-horizon financial tasks. *arXiv preprint arXiv:2604.10015*.
- Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. *arXiv preprint arXiv:2404.01744*.
- Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. *FinQA: A dataset of numerical reasoning over financial data*. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711.
- B. Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.
- Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.
- Georgi Gerganov. 2023. *llama.cpp*.
- Tom Gunter, Zirui Wang, Chong Wang, Ruoming Huang, Yandong Zhang, Xianzhi Liang, and Apple Intelligence Team. 2024. Apple intelligence foundation language models. *arXiv preprint arXiv:2407.21075*.
- Polaris Jhandi, Owais Kazi, Shreyas Subramanian, and Neel Sendas. 2025. Small language models for efficient agentic tool calling: Outperforming large models with targeted fine-tuning. *arXiv preprint arXiv:2512.15943*.
- Ishan Kavatthekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. *Small models, big tasks: An exploratory empirical study on small language models for function calling*. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 1117–1126.
- LG AI Research. 2025. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. *arXiv preprint arXiv:2507.11407*.
- Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Zhang, and Yunxin Liu. 2024. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.
- Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Ying Liu, and Xiaojun Hu. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587*.
- Liquid AI. 2026. *Introducing LFM2.5: The next generation of on-device AI*. Liquid AI Blog.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunsong Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024a. *MobileLLM: Optimizing sub-billion parameter language models for on-device use cases*. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32431–32454.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Tan, Joey Tianyi Zhou, Philip S. Yu, and Steven C. H. Hoi. 2024b. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37.
- Helen Nissenbaum. 2004. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158.
- OpenBMB. 2026. *MiniCPM5-1B: A SOTA 1B on-device LLM*. GitHub repository. Accessed: 2026-08-04.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. *The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models*. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 48371–48392.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. *Gorilla: Large language model connected with massive APIs*. *arXiv preprint arXiv:2305.15334*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Siyuan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, and Zhiyuan Liu. 2024. *ToolLLM: Facilitating large language models to master 16000+ real-world APIs*. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.
- Qwen Team. 2026. *Qwen3.5: Towards native multi-modal agents*. Qwen Blog.

668 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta
669 Raileanu, Maria Lomeli, Eric Hambro, Luke Zettle-
670 moyer, Nicola Cancedda, and Thomas Scialom. 2023.
671 Toolformer: Language models can teach themselves
672 to use tools. In *Advances in Neural Information Pro-*
673 *cessing Systems*, volume 36.

674 Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li,
675 Weiming Lu, and Yueting Zhuang. 2023. Hugging-
676 GPT: Solving AI tasks with ChatGPT and its friends
677 in Hugging Face. In *Advances in Neural Information*
678 *Processing Systems*, volume 36.

679 Yewei Song, Xunzhu Tang, Cedric Lothritz, Saad Ezzini,
680 Jacques Klein, Tegawendé F. Bissyandé, Andrey
681 Boytsov, Ulrick Ble, and Anne Goujon. 2025. [Call-](#)
682 [Navi: A challenge and empirical study on LLM func-](#)
683 [tion calling and routing](#). In *Proceedings of the 29th*
684 *International Conference on Evaluation and Assess-*
685 *ment in Software Engineering*.

686 Robin Staab, Mark Vero, Mislav Balunović, and Mar-
687 tin Vechev. 2024. Beyond memorization: Violating
688 privacy via inference with large language models. In
689 *Proceedings of the 12th International Conference on*
690 *Learning Representations*.

691 Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han,
692 Qiao Liang, and Le Sun. 2023. ToolAlpaca: Gener-
693 alized tool learning for language models with 3000
694 simulated cases. *arXiv preprint arXiv:2306.05301*.

695 Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao,
696 Qi Cai, and Ziyuan Ling. 2024. On-device language
697 models: A comprehensive review. *arXiv preprint*
698 *arXiv:2409.00088*.

699 Hongyang Yang, Xiao-Yang Liu, and Christina Dan
700 Wang. 2023. [FinGPT: Open-source financial large](#)
701 [language models](#). In *International Symposium on*
702 *Large Language Models for Financial Services (Fin-*
703 *LLM 2023) at IJCAI 2023*.

704 Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo
705 Sun, and Yue Zhang. 2024. A survey on large lan-
706 guage model (LLM) security and privacy: The good,
707 the bad, and the ugly. *High-Confidence Computing*,
708 4(2):100211.

709 Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi
710 Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng
711 Chen, Yong Liu, and Chi Zhang. 2026. FinMCP-
712 Bench: Benchmarking LLM agents for real-world
713 financial tool use under the model context protocol.
714 *arXiv preprint arXiv:2603.24943*.

715 A Complexity Scoring Features

Feature	Range	Description
n_p	0+	Number of expected parameters
d_{date}	0–4	Date complexity: none / year / month / range / specific day
d_{cat}	0–2	Category distance: exact / alias / semantic
d_{rel}	0–1	Relative time expression (“past X months”)
d_{multi}	0–2	Compound parameter groups
d_{abbr}	0–1	Abbreviated year formats (“’24”)
d_{hol}	0–2	Holiday/calendar knowledge required

Table 6: Seven features of the complexity scoring function (Eq. 1).

716 B Known Category Vocabulary

717 The post-hoc parameter validation layer (Section 4.3) and the LLM prompts rely on a known cate-
718 gory vocabulary defined dynamically or in static schema files. Table 7 lists the 11 major categories
719 defined in the application alongside the mapped subcategories/aliases from the system configuration
720 (`categories.json`) that are passed in the `{metadata}` block.

Major Category	Mapped Minor Categories / Subcategories
Housing and Utilities	housing, internet bill, electricity bill, gas bill, water & sewage bill, phone bill
Food	grocery, snacks, cafe, café, coffee, bento, beverage, combinini meal, dining, eating with friend
Transportation	commute, ride share, tokyo metro, Tokyo Metro, flight tickets, cable car, bus, shinkansen, car rental, taxi
Accommodation	stay
Fitness	supplements, shoes, sports event, sports watch, sports clothing, sports rental, gym, sports equipment, basketball game, football game, futsal game
Souvenirs/Gifts/Treats	souvenirs, treat, gift, souvenirs/gifts/treats
Household and Clothing	clothing, household
Entertainment	entertainment, nomikai, activities, Activities, arcades & karaoke, Arcades & Karaoke, events & venues, Events & Venues
Miscellaneous	medicines, personal care, misc, help, charity, donation, entrance fees, park entrance fees, healthcare
Education	tuition, books, exam fees, notebooks, textbook
Electronics and Furniture	furniture, electronics

Table 7: Known category vocabulary from `categories.json`.

721 C Sample Test Cases

ID	Query	Tool	Expected Params	Tier
TS01	Can ya show spending on food for past 6 months?	<code>plot_time_series</code>	<code>category=Food, months=6</code>	L2
DI06	show breakdown of expenses for june 21 2024	<code>plot_distribution</code>	<code>year=2024, month=6, day=21</code>	L2
CP24	compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026	<code>plot_comparison_bars</code>	<code>category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4</code>	L3
CT07	sum total spent on metro for july 2024?	<code>calculate_total</code>	<code>category=tokyo metro, year=2024, month=7</code>	L2
TP06	top 5 expenses past month? disregard rent	<code>get_top_expenses</code>	<code>n=5, months=1, ignore_rent=true</code>	L2

Table 8: Sample test cases. Note informal language, abbreviations, and semantic mappings (“metro”→“tokyo metro”, “groceries”→“grocery”).

722 **D Qualitative Error Examples**

723 Table 9 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

Error Type	Query	Expected	Model Output
PARAM_MISSING	“Show spending trend for the last 3 months”	{months: 3}	{ } (months omitted in the relative-time case)
PARAM_VALUE_WRONG	“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”	{em1: 4, em2: 4}	{em1: 11, em2: 11} (confuses start month for end month in cross-year range)
PARAM_EXTRA	“How much did I spend on food?”	{category: Food}	{category: Food, months: 1} (spurious time filter restricts scope to last month, silently returning an incorrect total)

Table 9: Representative failure cases for Qwen3.5-2B (dual mode). PARAM_EXTRA is the most dangerous failure mode in financial contexts: the spurious months=1 filter silently restricts “total food spending” to “last month’s food spending”, returning a plausible but incorrect figure that a user may act on without realising the scope was changed.

724 E Supplementary Figures

725 Figure 4 visualises the full error distribution breakdown. Figure 5 presents the model $\times$ category Task
 726 Accuracy heatmap. Figure 6 shows the per-category footprint across all evaluated models. Figure 7
 727 illustrates the metrics improvement provided by the post-hoc validation layer. Figure 8 shows the token
 728 efficiency trade-off between architectures. Figures 9 and 10 provide intermediate views of the real-world
 729 case study (Section 8).

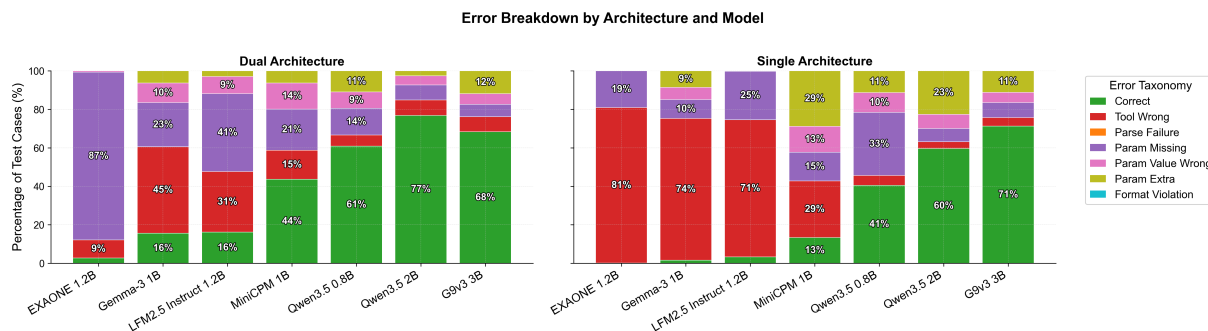


Figure 4: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B produces 22.6% `PARAM_EXTRA` errors (spurious parameters that could lead to silently incorrect financial outputs). Dual-agent eliminates most of these, consolidating residual errors into the more predictable `PARAM_MISSING` category.

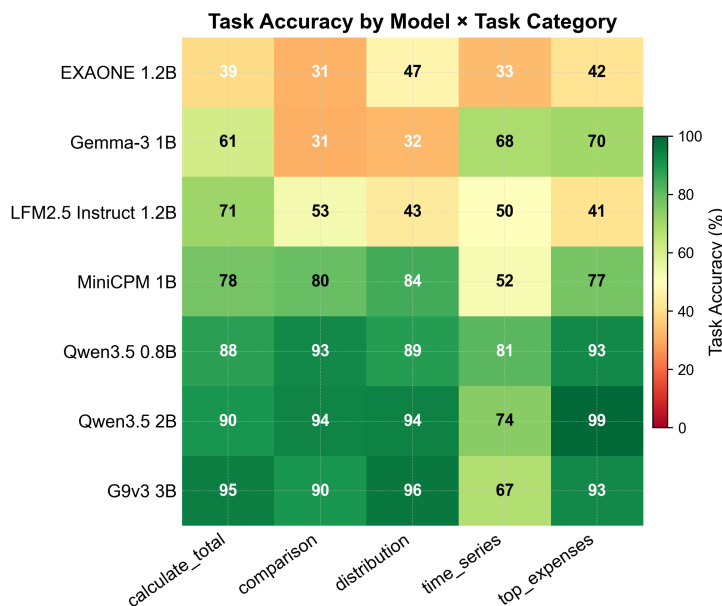


Figure 5: Task Accuracy (%) by model $\times$ tool category in dual mode. Qwen3.5 models achieve $>74\%$ across all categories, while weaker models show marked gaps in comparison and time series tasks.

Per-Model Task Footprint: Architecture Comparison

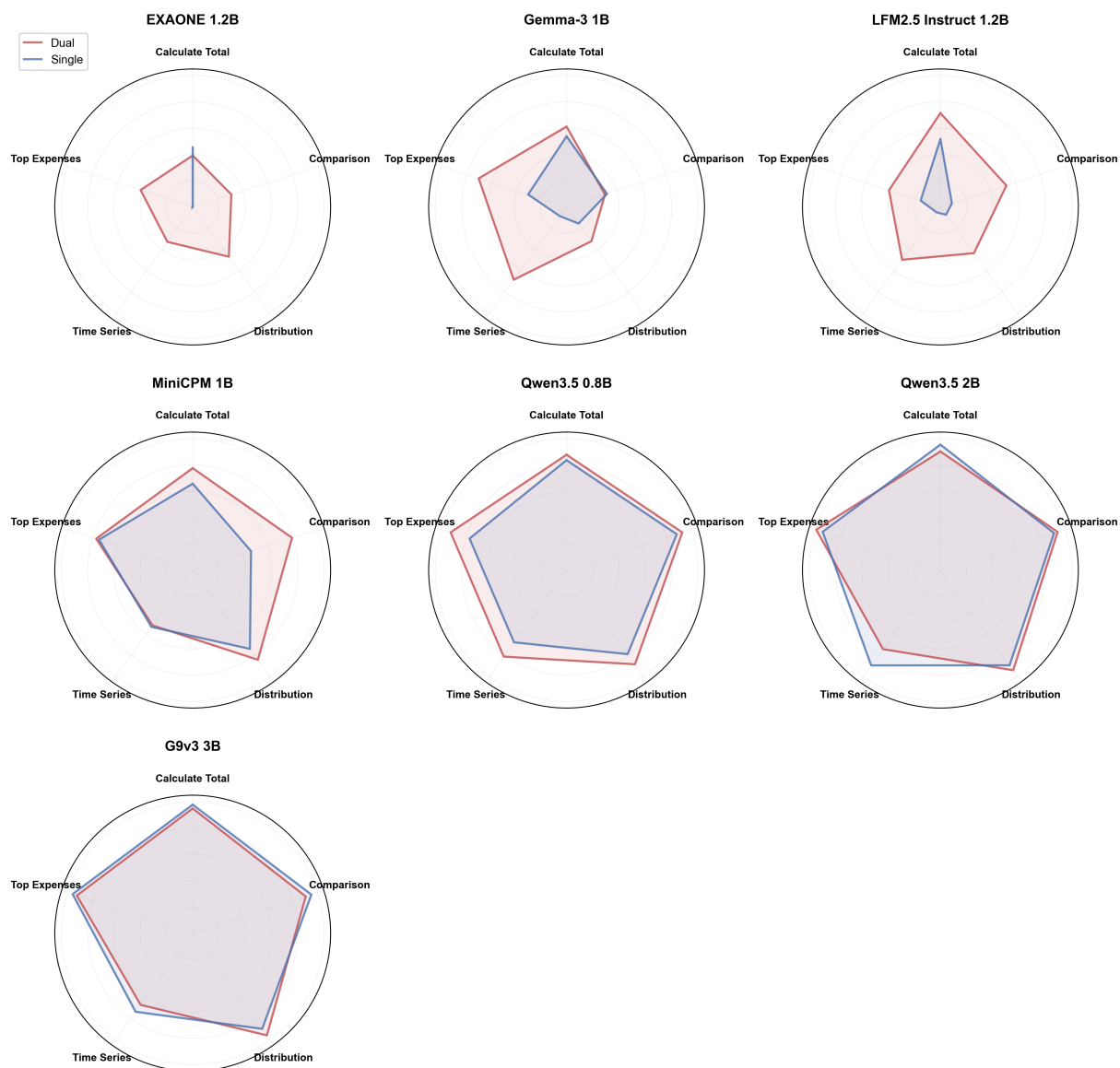


Figure 6: Per-model task footprint across all five task categories (Single vs. Dual mode). The dual-agent architecture provides near-universal improvement across models and tasks.

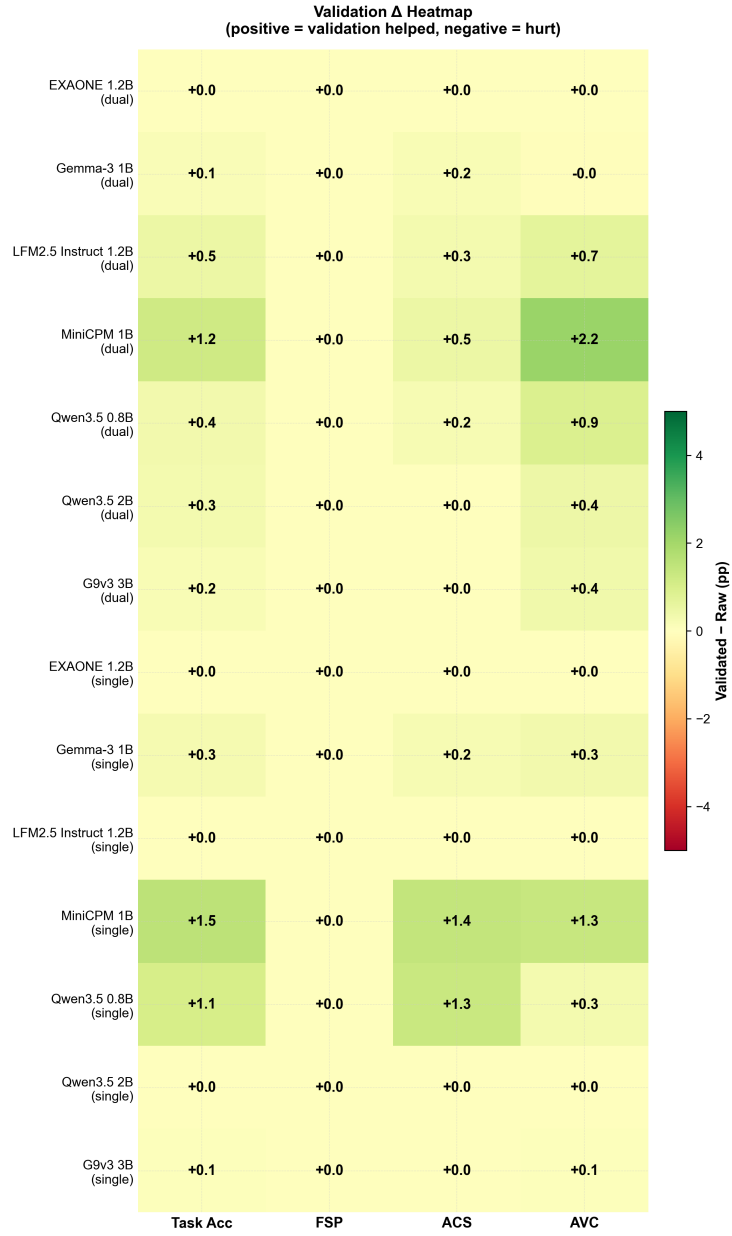


Figure 7: Validation Δ Heatmap, showing absolute percentage point improvement across metrics after applying post-hoc fuzzy matching.

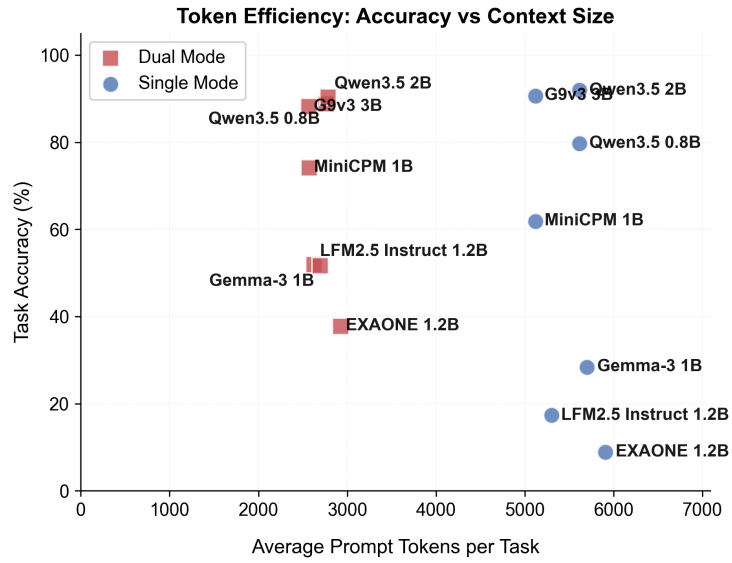


Figure 8: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red squares) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue circles), demonstrating superior context utilisation.

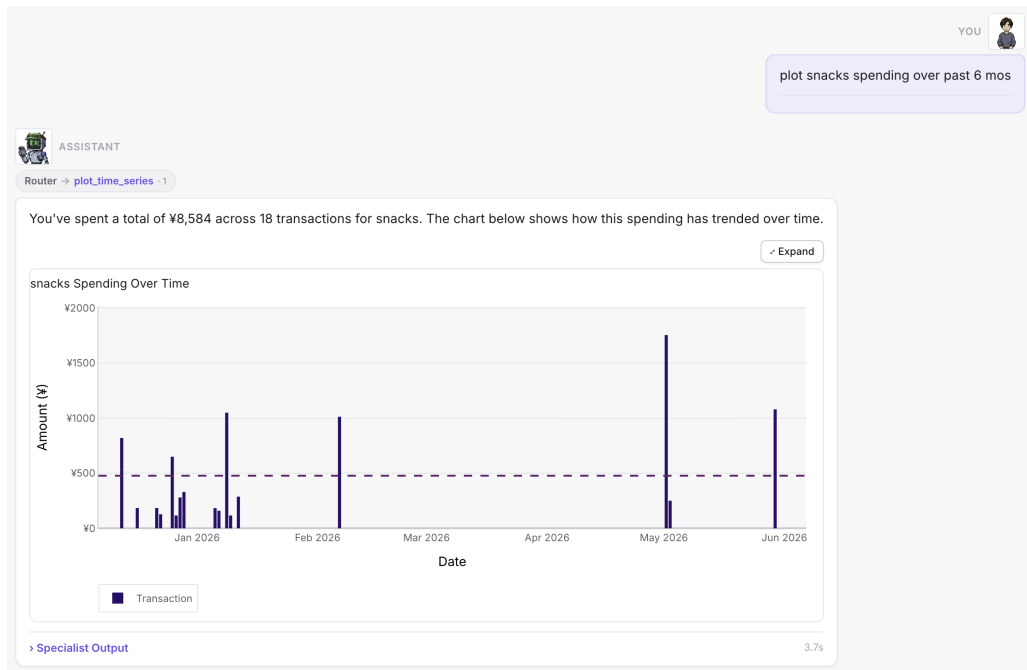


Figure 9: Case study intermediate view: six months *after* the participant decided to reduce snack spending (queried June 2026). Only 18 transactions totalling ¥8,584 remain over six months, with long gaps between purchases, in stark contrast to the dense pattern in Figure 3a.

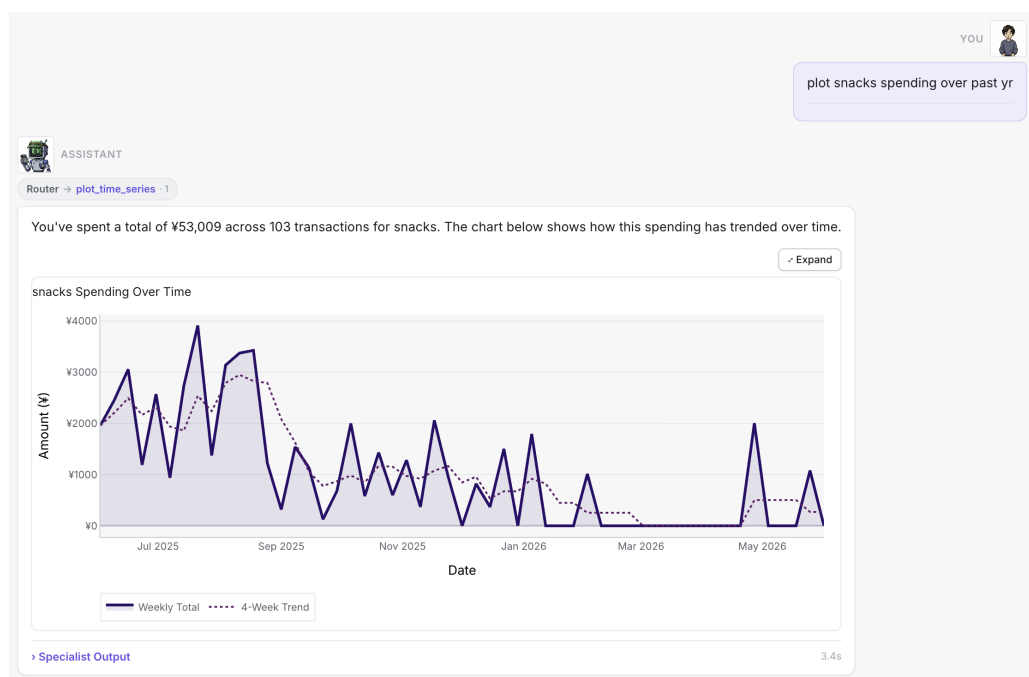


Figure 10: Case study full-year view: weekly snack spending over the past year (queried June 2026). The 4-week moving average (dotted line) shows a clear downward trend from peaks of ¥3,000+ in mid-2025 to near-zero by early 2026, illustrating the sustained behaviour change following the initial alert in January 2026.

730 F Prompt Templates

731 The on-device prompts utilised for routing and parameter extraction tasks are detailed below.

732 F.1 Router Prompt

733 The router receives a focused intent-classification prompt:

```
734 You are an expert intent classifier for an expense analysis assistant.
735 Your job is to determine which ONE tool is best suited to answer the user's
736 question.
737
738 ## Available Tools
739
740 1. **`plot_time_series`** (ID: 1)
741 - Use when user asks about: trends, spending over time, "past X months",
742   "since 2023", date ranges.
743 - Keywords: trend, over time, plot, show spending, changed, since.
744 - NOTE: if user says "trend" or "over time" WITH "exclude rent", still
745   use this tool.
746
747 2. **`plot_distribution`** (ID: 2)
748 - Use when user asks for: breakdown, distribution, proportions, pie
749   chart, "how is X split", "what does X look like".
750 - Keywords: breakdown, distribution, distro, pie chart, split, look like.
751 - NOTE: if user says "breakdown" or "split" WITH "exclude rent", still
752   use this tool.
753
754 3. **`plot_comparisonBars`** (ID: 3)
755 - Use ONLY when comparing TWO different time periods (e.g. "Dec 2024 vs
756   Dec 2025" or ranges like "Jan-Jun 2024 vs Jan-Jun 2025").
757 - REQUIRES two distinct periods. If the user only mentions one month or
758   one year, DO NOT use this tool.
759 - Keywords: compare, contrast, vs, versus, difference between.
760
761 4. **`calculate_total`** (ID: 4)
762 - Use when user asks for: simple totals, sums, specific amounts "how
763   much did I spend".
764 - Keywords: how much, total, sum, cost.
765
766 5. **`get_top_expenses`** (ID: 5)
767 - Use when user asks for: biggest/largest expenses, top X items, most
768   expensive.
769 - Keywords: biggest, largest, top, most expensive, highest.
770 - NOTE: if user says "top X" WITH "exclude rent", still use this tool.
771
772 ## Output Format
773 Output ONLY the tool number (1-5). Do not output anything else, NO
774 MARKDOWN, NO BACKTICKS, NO EXPLANATION, NO CODE!
775
776 ## WRONG outputs (NEVER do this):
777 [X] I think the best tool is 4
778 [X] `4`
779 [X] Based on the query, I would use 1 because...
780 [X] ```4```
781 [X] The answer is: 2
782
783 ## CORRECT outputs:
784 [v] 4
785 [v] 1
786
787 ## Examples
788 User: "How much did I spend on food?"
789 Output: 4
790
791 User: "Total cost of internet for the last 3 months"
792 Output: 4
793
794 User: "How much did I spend in the last 6 months without rent?"
795 Output: 4
796
797 User: "Show me my spending trend for groceries"
798 Output: 1
799
800 User: "How has my rent changed over the last 6 months?"
801 Output: 1
802
803 User: "Show spending trend for past 4 months exclude rent tho"
804 Output: 1
805
806 User: "Compare food spending in 2024 and 2025"
807 Output: 3
808
809 User: "Difference in spending between Dec 2023 and Dec 2024"
810 Output: 3
811
812 User: "Compare overall spend in '24 vs '25 (exclude rent tho)"
```

```

813 Output: 3
814
815 User: "Compare dining between Jan-Jun 2024 and Jan-Jun 2025"
816 Output: 3
817
818 User: "Contrast spending from Nov 2024 to Feb 2025 vs Nov 2025 to Feb 2026"
819 Output: 3
820
821 User: "What were my biggest expenses?"
822 Output: 5
823
824 User: "Show me my top 5 largest purchases this year"
825 Output: 5
826
827 User: "Top 10 items without rent"
828 Output: 5
829
830 User: "Pie chart of transportation"
831 Output: 2
832
833 User: "How is my spending split in 2023?"
834 Output: 2
835
836 User: "Show me my spending breakdown excluding rent for the past 6 months"
837 Output: 2
838
839 User: "what does my electronics spending look like for jan 2025?"
840 Output: 2
841
842 User: "share distro of household expenses"
843 Output: 2

```

844 F.2 Specialist Base Template

845 Each specialist prompt instantiates a shared template defining output structure and relative date handling.
846 The {metadata} placeholder is populated dynamically with the categories present in the user-uploaded
847 dataset:

```

848 You are an automated API parameter extractor. Your ONLY purpose is to extract
849 parameters from the user's request and output a SINGLE JSON object containing
850 those parameters.
851
852 ## CRITICAL RULES (READ CAREFULLY)
853 1. DO NOT write any Python code or function calls. DO NOT import anything.
854 2. DO NOT wrap the output in markdown. NO backticks (`).
855 3. Output EXACTLY and ONLY a JSON object representing the parameters.
856    E.g. {"category": "Food", "months": 6}
857 4. Use EXACT category names from the metadata below. If not an exact match,
858    map it to the closest valid category (e.g. 'groceries' -> 'grocery',
859    'dining out' -> 'dining').
860
861 ## Date Rules
862 - 'months=N': RELATIVE duration (e.g. "past month" -> months=1, "last 3
863   months" -> months=3, "past year" -> months=12).
864 - Fractional years: convert to months. "past 0.5 years" -> months=6, "past
865   2.5 years" -> months=30.
866 - Abbreviated years: '24 = 2024, 24 = 2024, ALWAYS output the full 4-digit
867   year.
868 - 'year=YYYY': specific full calendar year.
869 - 'year=YYYY, month=M': specific calendar month (e.g. "Jan 2024").
870 - 'year=YYYY, month=M, day=D': specific calendar date.
871 - 'start_year=YYYY, end_year=YYYY': specific year range.
872 - 'start_year=YYYY, start_month=M, end_year=YYYY, end_month=M': specific
873   month-to-month range.
874 - For comparing two ranges (e.g., comparing Jan-Jun 2024 vs Jan-Jun 2025),
875   use comparison range parameters: 'y1=2024, sm1=1, em1=6, y2=2025, sm2=1,
876   em2=6'.
877 - For comparing cross-year ranges (e.g., Nov 2024-Jan 2025 vs Nov 2025-Jan
878   2026), use 'ey1' and 'ey2': 'y1=2024, sm1=11, ey1=2025, em1=1, y2=2025,
879   sm2=11, ey2=2026, em2=1'.
880 - CRITICAL: For relative queries like "past month", "last month", "last 6
881   months", ALWAYS use 'months=N'. Do NOT use 'month=M'.
882 - Pick ONLY ONE time filter. Do not mix 'months' with 'year'.
883
884 ## Available Parameters
885 {parameters}
886
887 ## Examples
888 {examples}
889
890 ## Context
891 ```
892 {metadata}
893 ```
894 Currency: JPY
895 Today: {current_date}
896

```

897 FINAL REMINDER: Output ONLY the JSON object. NO MARKDOWN, NO BACKTICKS,
898 NO EXPLANATION, NO CODE!

899 F.3 Specialist Tool Prompts

900 Each tool-specific specialist appends its unique parameter definitions and few-shot examples to the base
901 template. The precise specifications for each tool are detailed below.

902 F.3.1 plot_time_series

```
903 Parameters:
904   category (str, optional), year (int, optional), month (int, optional),
905   start_year (int, optional), start_month (int, optional),
906   end_year (int, optional), end_month (int, optional),
907   months (int, optional), ignore_rent (bool, optional, defaults to false)
908
909 ## Examples
910 Q: "How much did I spend on futsal for the past 6 months?"
911 {"category": "futsal game", "months": 6}
912
913 Q: "Show me food spending from 2023 to 2025"
914 {"category": "Food", "start_year": 2023, "end_year": 2025}
915
916 Q: "Plot spending on snacks from oct 2024 to dec 2025"
917 {"category": "snacks", "start_year": 2024, "start_month": 10, "end_year": 2025, "end_month": 12}
918
919 Q: "Can ya plot spending on futsal on dec 2024?"
920 {"category": "futsal game", "year": 2024, "month": 12}
921
922 Q: "Gym expenses in 2024?"
923 {"category": "gym", "year": 2024}
924
925 Q: "Show me my spending trend for the last 3 months"
926 {"months": 3}
927
928 Q: "plot spending from june 2023 to dec 2025"
929 {"start_year": 2023, "start_month": 6, "end_year": 2025, "end_month": 12}
930
931 Q: "Trend of electricity bills since 2022"
932 {"category": "electricity bill", "start_year": 2022}
933
934 Q: "How has my transportation spending changed over time?"
935 {"category": "Transportation"}
936
937 Q: "Show spending on combinis for past year"
938 {"category": "combinini meal", "months": 12}
939
940 Q: "Show spending trend for past 4 months exclude rent tho"
941 {"months": 4, "ignore_rent": true}
942
943 Q: "plot spend on cafes for past 2.5 yrs"
944 {"category": "cafe", "months": 30}
945
946 Q: "show my ride share costs from '24 to 26"
947 {"category": "ride share", "start_year": 2024, "end_year": 2026}
948
949 Q: "show electronics spending since 2023"
950 {"category": "electronics", "start_year": 2023}
```

951 F.3.2 plot_distribution

```
952 Parameters:
953   category (str, optional), year (int, optional), month (int, optional),
954   day (int, optional), start_year (int, optional), start_month (int, optional),
955   end_year (int, optional), end_month (int, optional), months (int, optional),
956   ignore_rent (bool, optional, defaults to false)
957
958 ## Examples
959 Q: "Show me a breakdown of my food expenses in 2024"
960 {"category": "Food", "year": 2024}
961
962 Q: "Pie chart of all expenses from Nov 2024 to Feb 2025"
963 {"start_year": 2024, "start_month": 11, "end_year": 2025, "end_month": 2}
964
965 Q: "Show me my spending breakdown for last month"
966 {"months": 1}
967
968 Q: "Show me my spending breakdown for groceries for past 6 months"
969 {"category": "Grocery", "months": 6}
970
971 Q: "show spend distribution for 2026 feb"
972 {"year": 2026, "month": 2}
973
974 Q: "show breakdown of all expenses from 2024 to 2025"
975 {"start_year": 2024, "end_year": 2025}
976
```

```

977 Q: "Show me a breakdown of expenses for Dec 2024 (exclude rent tho)"
978 {"year": 2024, "month": 12, "ignore_rent": true}
979
980 Q: "Show spending breakdown for 2023/06/22"
981 {"year": 2023, "month": 6, "day": 22}
982
983 Q: "Spending distribution for the last 3 months"
984 {"months": 3}
985
986 Q: "Breakdown of transportation spending from 2022 to 2024"
987 {"category": "Transportation", "start_year": 2022, "end_year": 2024}
988
989 Q: "How did I split my money between categories in 2023?"
990 {"year": 2023}
991
992 Q: "Show me my spending breakdown excluding rent for the past 6 months"
993 {"months": 6, "ignore_rent": true}
994
995 Q: "Breakdown of fitness spending for 2025"
996 {"category": "Fitness", "year": 2025}
997
998 Q: "Breakdown of my spending since 2024"
999 {"start_year": 2024}
1000
1001 Q: "show breakdown of all expenses for mar 2026 (exclude rent tho)"
1002 {"year": 2026, "month": 3, "ignore_rent": true}
1003
1004 Q: "what does my electronics and furniture spending look like for jan 2025?"
1005 {"category": "Electronics and Furniture", "year": 2025, "month": 1}

```

1006 F3.3 plot_comparison_bars

```

1007 Parameters:
1008     category (str, optional), y1 (int, optional), m1 (int, optional),
1009     d1 (int, optional), y2 (int, optional), m2 (int, optional),
1010     d2 (int, optional), sm1 (int, optional), em1 (int, optional),
1011     sm2 (int, optional), em2 (int, optional), ey1 (int, optional),
1012     ey2 (int, optional), ignore_rent (bool, optional, defaults to false)
1013
1014 ## Examples
1015 Q: "Compare food spending in 2024 vs 2025"
1016 {"category": "Food", "y1": 2024, "y2": 2025}
1017
1018 Q: "Compare dining Jan 2024 vs Jan 2025"
1019 {"category": "dining", "y1": 2024, "m1": 1, "y2": 2025, "m2": 1}
1020
1021 Q: "How does my spend on snacks compare between jan 2024 and jan 2026?"
1022 {"category": "snacks", "y1": 2024, "m1": 1, "y2": 2026, "m2": 1}
1023
1024 Q: "Compare total spending 2022 vs 2023"
1025 {"y1": 2022, "y2": 2023}
1026
1027 Q: "Compare overall expenses from jan 2025 vs jan 2026"
1028 {"y1": 2025, "m1": 1, "y2": 2026, "m2": 1}
1029
1030 Q: "Compare transportation on 21 July 2024 vs 21 July 2025"
1031 {"category": "Transportation", "y1": 2024, "m1": 7, "d1": 21, "y2": 2025, "m2": 7, "d2": 21}
1032
1033 Q: "Compare spend on electricity 2024 vs 2025"
1034 {"category": "electricity bill", "y1": 2024, "y2": 2025}
1035
1036 Q: "Compare gym spending Nov 2024 vs Nov 2025"
1037 {"category": "gym", "y1": 2024, "m1": 11, "y2": 2025, "m2": 11}
1038
1039 Q: "Compare my spending in 2024 vs 2025 excluding rent"
1040 {"y1": 2024, "y2": 2025, "ignore_rent": true}
1041
1042 Q: "compare education expenses '24 vs '25"
1043 {"category": "Education", "y1": 2024, "y2": 2025}
1044
1045 Q: "compare household expenses for april '24 vs apr 2025"
1046 {"category": "household", "y1": 2024, "m1": 4, "y2": 2025, "m2": 4}
1047
1048 Q: "Compare dining between Jan-Jun 2024 and Jan-Jun 2025"
1049 {"category": "dining", "y1": 2024, "sm1": 1, "em1": 6, "y2": 2025, "sm2": 1, "em2": 6}
1050
1051 Q: "compare electricity from jan-mar '24 vs jan-mar '25"
1052 {"category": "electricity bill", "y1": 2024, "sm1": 1, "em1": 3, "y2": 2025, "sm2": 1, "em2": 3}
1053
1054 Q: "Contrast overall spend from Nov 2024 to Feb 2025 vs Nov 2025 to Feb 2026"
1055 {"y1": 2024, "sm1": 11, "ey1": 2025, "em1": 2, "y2": 2025, "sm2": 11, "ey2": 2026, "em2": 2}
1056
1057 Q: "compare total spend on new years eve 2024 vs for same day on 2025"
1058 {"y1": 2024, "m1": 12, "d1": 31, "y2": 2025, "m2": 12, "d2": 31}

```

1059 F3.4 calculate_total

```

1060 Parameters:

```

```

1061     category (str, optional), year (int, optional), month (int, optional),
1062     day (int, optional), start_year (int, optional), start_month (int, optional),
1063     end_year (int, optional), end_month (int, optional), months (int, optional),
1064     ignore_rent (bool, optional, defaults to false)
1065
1066 ## Examples
1067 Q: "How much did I spend on groceries in Dec 2024?"
1068 {"category": "grocery", "year": 2024, "month": 12}
1069
1070 Q: "Total spending from Oct 2024 to March 2025"
1071 {"start_year": 2024, "start_month": 10, "end_year": 2025, "end_month": 3}
1072
1073 Q: "What is my total spending in 2025?"
1074 {"year": 2025}
1075
1076 Q: "How much did I spend on food in past month?"
1077 {"category": "Food", "months": 1}
1078
1079 Q: "Total cost of electricity in 2023"
1080 {"category": "electricity bill", "year": 2023}
1081
1082 Q: "Total spent on 15 Jan 2024?"
1083 {"year": 2024, "month": 1, "day": 15}
1084
1085 Q: "How much spent on rent in the last 6 months?"
1086 {"category": "rent", "months": 6}
1087
1088 Q: "Sum of all transportation expenses in 2025"
1089 {"category": "Transportation", "year": 2025}
1090
1091 Q: "Total spent on combini food for 2025?"
1092 {"category": "combin meal", "year": 2025}
1093
1094 Q: "How much did I spend in the last 6 months without rent?"
1095 {"months": 6, "ignore_rent": true}
1096
1097 Q: "Total spent on dining since 2024"
1098 {"category": "dining", "start_year": 2024}
1099
1100 Q: "whats the total spend on donations from '23 to 2026?"
1101 {"category": "donation", "start_year": 2023, "end_year": 2026}
1102
1103 Q: "Excluding rent, what was my total spend in 2025?"
1104 {"year": 2025, "ignore_rent": true}

```

1105 F3.5 get_top_expenses

```

1106 Parameters:
1107     n (int, optional, defaults to 10), category (str, optional),
1108     year (int, optional), month (int, optional), day (int, optional),
1109     start_year (int, optional), start_month (int, optional),
1110     end_year (int, optional), end_month (int, optional),
1111     months (int, optional), min_amount (int, optional),
1112     ignore_rent (bool, optional, defaults to false)
1113
1114 ## Examples
1115 Q: "What were my biggest expenses in Dec 2024?"
1116 {"n": 10, "year": 2024, "month": 12}
1117
1118 Q: "Top 5 food expenses in 2024?"
1119 {"n": 5, "category": "Food", "year": 2024}
1120
1121 Q: "Top 10 food expenses in june 2025?"
1122 {"n": 10, "category": "Food", "year": 2025, "month": 6}
1123
1124 Q: "Top 5 expenses for 26 june 2024?"
1125 {"n": 5, "year": 2024, "month": 6, "day": 26}
1126
1127 Q: "Top expenses from July 2023 to Dec 2024"
1128 {"start_year": 2023, "start_month": 7, "end_year": 2024, "end_month": 12}
1129
1130 Q: "Show my top 10 expenses from 2023 to 2025"
1131 {"n": 10, "start_year": 2023, "end_year": 2025}
1132
1133 Q: "Top 10 expenses of past month?"
1134 {"n": 10, "months": 1}
1135
1136 Q: "What are my top expenses excluding rent?"
1137 {"n": 10, "ignore_rent": true}
1138
1139 Q: "Show my top 3 largest transactions in 2023"
1140 {"n": 3, "year": 2023}
1141
1142 Q: "What were the biggest expenses over the last 3 months, without rent?"
1143 {"n": 10, "months": 3, "ignore_rent": true}
1144
1145 Q: "Top 7 futsal expenses for 2025?"

```

1146 {"n": 7, "category": "futsal game", "year": 2025}
1147
1148 Q: "Top 5 expenses last month, exclude rent"
1149 {"n": 5, "months": 1, "ignore_rent": true}
1150
1151 Q: "What were my biggest expenses since 2023?"
1152 {"n": 10, "start_year": 2023}
1153
1154 Q: "show me top 10 expenses from jan 2025 to june '25"
1155 {"n": 10, "start_year": 2025, "start_month": 1, "end_year": 2025, "end_month": 6}
1156
1157 Q: "tell me the top 15 expenses for 2025, ignore rent plz"
1158 {"n": 15, "year": 2025, "ignore_rent": true}